

Pandas → data handling
DataFrame → table in Python

.csv → comma separated → read_csv()
.xlsx → Excel → read_excel()
.txt → text → read_table() / read_csv()

delimiter → separator
\t → tab
NaN → missing value
na_values → specify values treated as missing
index_col → choose index column
sheet_name → choose Excel sheet
os → operating system / working directory

Pandas → Data analysis
DataFrame → 2D table
DataFrame → rows + columns

index → row labels
columns → column labels

shape → (rows, columns)
size → total elements
ndim → number of dimensions
memory_usage → memory in bytes

head() → first 5 rows
tail() → last 5 rows

at → label + single scalar
iat → integer position + single scalar
loc → label-based selection

shallow copy → shares reference
deep copy → independent copy

pandas → DataFrames
numpy → numerical operations
os → operating system / working directory

index_col=0 → first column becomes index

int64 → integers
float64 → decimal numbers

object → strings / mixed-type data
category → limited/fixed set of categorical values
df.dtypes

→ dtype of every column

df.dtypes.value_counts()

→ count of each dtype
(old lecture: `get_dtype_counts()`)

df.select_dtypes(include='number')

→ select numeric columns

df.select_dtypes(exclude='object')

→ exclude object columns

df.info()

→ concise DataFrame summary

df['column'].unique()

→ unique values

And the **big conceptual question**:

Why is a column that looks numeric sometimes `object`?

Because Pandas looks at the **actual values**, not what the column is supposed to represent. If it encounters strings, special characters, or mixed representations, the column may become `object`.

Import data

↓

Find missing-value representation

↓

Use info()

↓

Find problematic columns



Replace special values



Convert datatype using astype()



Choose category vs object appropriately



Clean Doors



Check missing values using isnull().sum()



Re-check dtypes



Code

Meaning

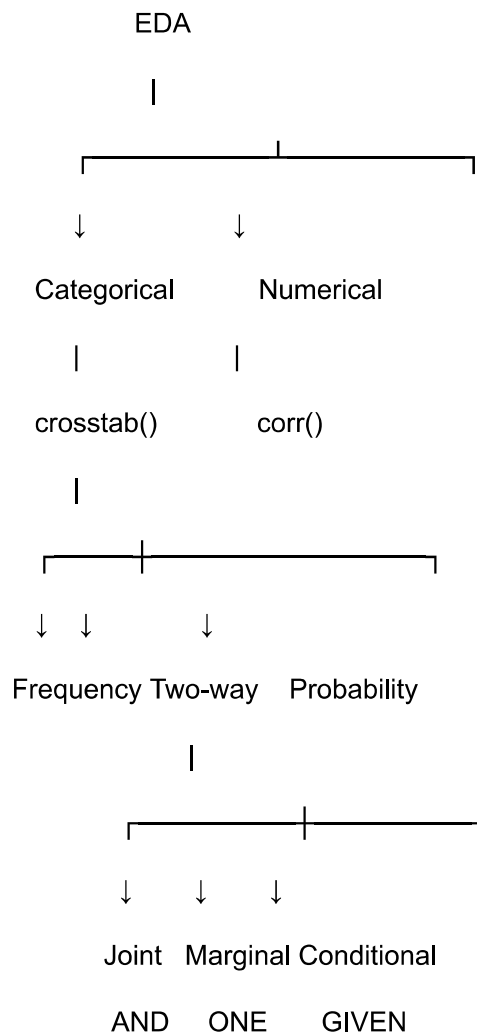
`pd.crosstab(A,B
)` Frequency

`normalize='inde
x'` Row-wise conditional probability

`normalize='all'` Joint probability

`normalize='columnwise'` Column-wise conditional probability
`mns'`

Remember the whole lecture like this:



If you remember just **these six words**, you'll already have the core:

Frequency = HOW MANY

Joint = AND

Marginal = ONE

Conditional = GIVEN

Correlation = RELATIONSHIP

crosstab = CATEGORICAL COUNTS

And one particularly important coding memory:

all → **joint**
index → **rows** → **row sum = 1**
columns → **columns** → **column sum = 1**

M-P-S-G-P

Matplotlib → 2D
Pandas → Easy
Seaborn → Statistical/attractive
Ggplot → Grammar of Graphics
Plotly → Interactive

```
import matplotlib.pyplot as plt
```

Scatter

```
plt.scatter(x, y)
```

```
plt.show()
```

Purpose: Relationship between **two numerical variables**.

Histogram

```
plt.hist(x)
```

```
plt.show()
```

Purpose: Frequency distribution of **numerical data**.

Bars = frequency

Bins = ranges

Bar

```
plt.bar(x, height)
```

```
plt.show()
```

Purpose: Frequency/count of **categorical data**.

height = height/value of bars

Pandas

```
import pandas as pd
```

Used for **DataFrames/data manipulation**.

NumPy

```
import numpy as np
```

Used for **numerical operations**.

Matplotlib

```
import matplotlib.pyplot as plt
```

Used for **visualization**.

Missing values

```
df.dropna()
```

One-line memory trick

SCATTER = RELATIONSHIP
HISTOGRAM = NUMERICAL FREQUENCY
BAR = CATEGORICAL FREQUENCY

Fuel type → categorical → Bar plot. ✓

Whereas:

Kilometres → numerical → Histogram. ✓

And:

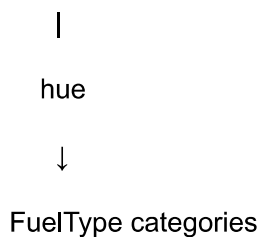
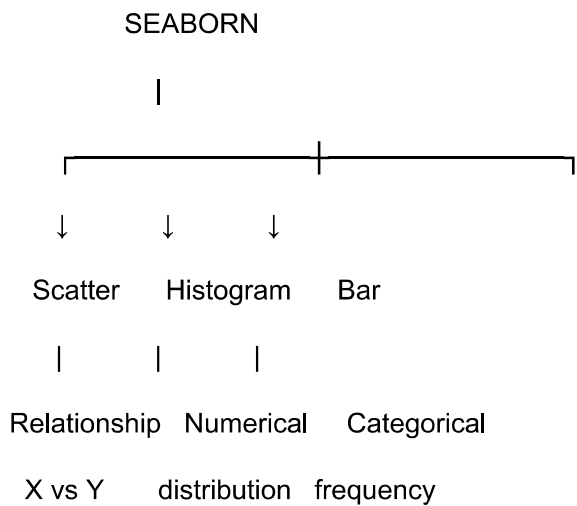
Age vs Price → two numerical variables → Scatter plot. ✓

If you have limited time tonight, **memorize these 10:**

- | # | Remember |
|----|---|
| 1 | <code>matplotlib.pyplot</code> → plotting |
| 2 | <code>plt.scatter(x,y)</code> → scatter plot |
| 3 | Scatter → relationship |
| 4 | Scatter → two numerical variables |
| 5 | <code>plt.hist(x)</code> → histogram |
| 6 | Histogram → numerical frequency distribution |
| 7 | Histogram ranges → bins |
| 8 | <code>plt.bar(x,height)</code> → bar plot |
| 9 | Bar plot → categorical frequency distribution |
| 10 | <code>plt.xticks()</code> → x-axis tick positions/labels |

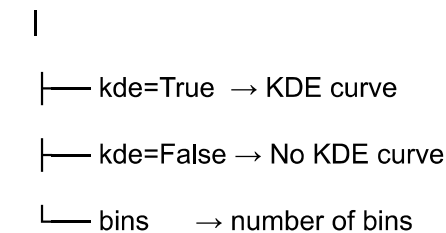
SUPER IMPORTANT MEMORY MAP

Before your exam, revise this:



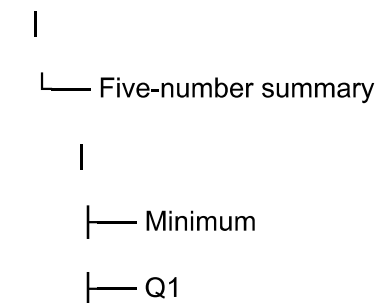
Then:

Histogram



And:

Box Plot



- └─ Median
- └─ Q3
- └─ Maximum

And:

Pairplot

- |
- └─ Pairwise relationships
- └─ Scatterplots
- └─ Univariate distributions
- └─ hue="FuelType"

Code	Meaning
<code>df.isna()</code>	Identify NaN
<code>df.isnull()</code>	Identify NaN
<code>df.isna().sum()</code>	Count missing values per column
<code>df.isnull().sum()</code>	Count missing values per column
<code>df.fillna(value)</code>	Fill missing values

`df.describe()` Descriptive statistics

`df['col'].value_counts()` Count unique values

`.value_counts().index[0]` Most frequent category

`df[df.isna().any(axis=1)]` Rows with ≥ 1 missing value



The 7 traps to absolutely remember

1 `isna()` does NOT fill data

`isna()` → detect

`fillna()` → fill

2 `isna()` returns Boolean

NaN → True

Value → False

3 `.sum()` after `isna()` counts missing values

`df.isna().sum()`

4 Numerical ≠ always mean

Numerical → mean OR median, depending on the data.

5 Categorical → mode

Mode = most frequent category.

6 `value_counts()` excludes NaN by default

And orders values by frequency in descending order.

7 `describe()` excludes NaN by default

It gives descriptive statistics rather than filling missing values.

One killer question to test yourself

Without looking above, complete this:

NaN → ? → ? → ?

The ideal answer is:

NaN → identify using `isna()/isnull()` → count using `.sum()` → fill using `fillna()`

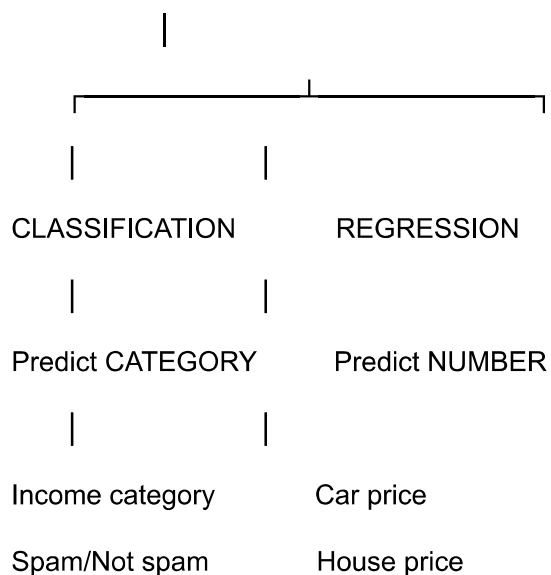
And for choosing what to fill with:

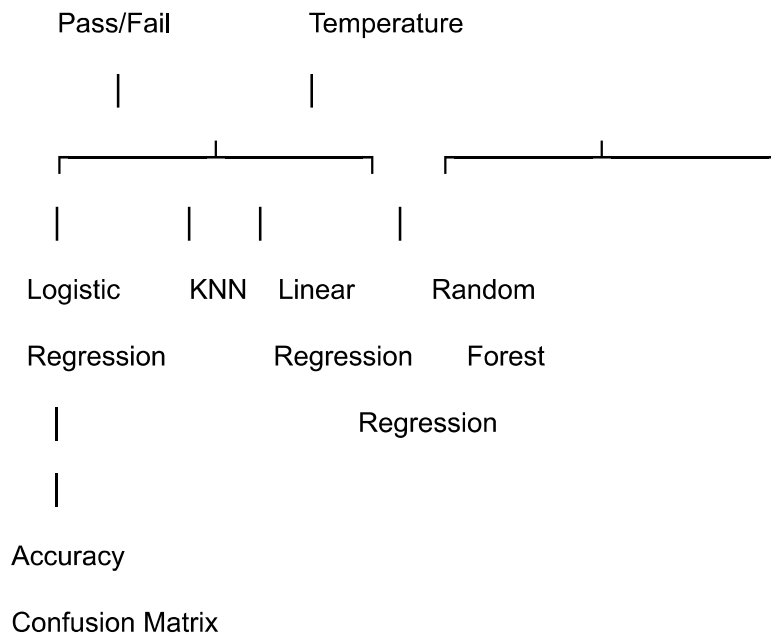
Numerical → mean/median

Categorical → mode

That chain is the **core of this entire lecture**.

MACHINE LEARNING





★ Now let's learn the vocabulary one by one

Word	Very simple meaning
Data	Information
Feature	Input information
Target	Thing we want to predict
X	Input/features
Y	Output/target

Classification	Predict category
Regression	Predict number
Binary	Two categories
Multiclass	More than two categories
Training	Teaching model using data
Testing	Checking model on unseen data
Encoding	Converting categories into numerical representation
<code>get_dummies()</code>	Creates dummy 0/1 columns
<code>dropna()</code>	Removes rows with missing values
<code>fit()</code>	Train the model
<code>predict()</code>	Make predictions
Correlation	Degree of linear relationship
Accuracy	Fraction of correct classifications

Confusion matrix Table of classification results

Residual Actual – predicted

RMSE Measure of regression prediction error

For today's exam, learn in this order

Don't try to memorize the entire Week 4 at once.

First, make these 5 things crystal clear:

1.

Classification = category

2.

Regression = number

3.

X = input, Y = output

4.

Training = learn, Testing = check

5.

`fit()` = train, `predict()` = predict

Then learn:

6.

`get_dummies()` = categorical data → 0/1 columns

7.

`dropna()` = remove missing values

8.

Accuracy → classification

9.

RMSE → regression

10.

Correlation does not automatically mean causation

CLASSIFICATION = CATEGORY

REGRESSION = NUMBER

BINARY = 2

MULTICLASS = MORE THAN 2

LOGISTIC REGRESSION = CLASSIFICATION

LINEAR REGRESSION = REGRESSION

X = INPUT

Y = OUTPUT

TRAIN = LEARN

TEST = CHECK

fit() = LEARN

predict() = PREDICT

get_dummies() = CATEGORIES → 0/1

dropna() = REMOVE MISSING VALUES

random_state = REPEATABLE RANDOMNESS

TP = YES predicted, YES actual

TN = NO predicted, NO actual

FP = YES predicted, NO actual

FN = NO predicted, YES actual

ACCURACY = CORRECT / TOTAL

RMSE = REGRESSION ERROR

LOWER RMSE = SMALLER ERROR

CORRELATION:

-1 → strong negative

0 → no/weak linear relationship

+1 → strong positive

CORRELATION ≠ CAUSATION

LINEAR EQUATION:

$$y = a + bx$$

$$\text{CHANGE IN } y = b \times \text{CHANGE IN } x$$

ZERO TRAINING ERROR

→ points lie on fitted hyperplane